

```

// constructor used when all dimensions specified
Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}

// constructor used when no dimensions specified
Box() {
    width = -1; // use -1 to
    indicate height = -1; // an
    uninitialized depth = -1; // box
}

// constructor used when cube is created
Box(double len) {
    width = height = depth = len;
}

// compute and return volume
double volume() {
    return width * height * depth;
}
}

// BoxWeight now fully implements all constructors.
class BoxWeight extends Box {
    double weight; // weight of box
    // construct clone of an object
    BoxWeight(BoxWeight ob) { // pass object to
        constructor super(ob);
        weight = ob.weight;
    }

    // constructor when all parameters are specified
    BoxWeight(double w, double h, double d, double m) {
        super(w, h, d); // call superclass constructor
        weight = m;
    }

    // default constructor
    BoxWeight()
    { super();
      weight = -1;
    }

    // constructor used when cube is created
    BoxWeight(double len, double m) {
        super(len);
        weight = m;
    }
}

class DemoSuper {
    public static void main(String args[]) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15,
        34.3); BoxWeight mybox2 = new BoxWeight(2, 3, 4,
        0.076); BoxWeight mybox3 = new BoxWeight(); //

        default BoxWeight mycube = new BoxWeight(3, 2);
        BoxWeight myclone = new BoxWeight(mybox1);
    }
}

```

~~double vol;~~

```
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);
System.out.println("Weight of mybox1 is " +
mybox1.weight); System.out.println();
vol = mybox2.volume();
System.out.println("Volume of mybox2 is " + vol);
System.out.println("Weight of mybox2 is " +
mybox2.weight); System.out.println();
vol = mybox3.volume();
System.out.println("Volume of mybox3 is " + vol);
System.out.println("Weight of mybox3 is " +
mybox3.weight); System.out.println();
vol = myclone.volume();
System.out.println("Volume of myclone is " + vol);
System.out.println("Weight of myclone is " +
myclone.weight); System.out.println();
vol = mycube.volume();
System.out.println("Volume of mycube is " + vol);
System.out.println("Weight of mycube is " +
mycube.weight); System.out.println();
}
}
```

Output:

```
Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3
Volume of mybox2 is 24.0
Weight of mybox2 is 0.076
Volume of mybox3 is -1.0
```

```
Weight of mybox3 is -1.0
Volume of myclone is 3000.0
Weight of myclone is 34.3
Volume of mycube is 27.0
Weight of mycube is 2.0
```

Calling members of super class using super

- o The second form of super acts somewhat like **this** keyword, except that it always refers to the super class of the sub class in which it is used.
- o The syntax is:
 - o **Super.member ;**
 - o Member can either be method or an instance variable

Program

```
// Using super to overcome name
hiding. class A {
int i;
}
// Create a subclass by extending class A.
```

```
class B extends A {
int i; // this i hides the i in A
B(int a, int b) {
```

```

super.i = a; // i in A
i = b; // i in B
}
void show() {
System.out.println("i in superclass: " + super.i);
System.out.println("i in subclass: " + i);
}
}
class UseSuper {
public static void main(String args[]) {
B subOb = new B(1, 2);
subOb.show();
}
}

```

Output:

i in superclass: 1
i in subclass: 2

When the constructor called:

Always the super class constructor will be executed first and sub class constructor will be executed last.

```

// Demonstrate when constructors are called.
// Create a super class.
class A {
A() {
System.out.println("Inside A's constructor.");
}
}
// Create a subclass by extending class A.
class B extends A {
B() {
System.out.println("Inside B's constructor.");
}
}

```

```

// Create another subclass by extending
B. class C extends B {
C() {
System.out.println("Inside C's constructor.");
}
}
class CallingCons {
public static void main(String args[]) {
C c = new C();
}
}

```

Output:

Inside	A's constructor
Inside	B's constructor
Inside	C's constructor

Using Final keyword:

- ☐ We can use final key word in **three** ways:
 - o Used to create equivalent of a named constant
 - ☐ Final datatype identifier =; o Used to prevent inheritance
 - ☐ Final class
 - o Used to avoid overloading
 - ☐ Final return type

Using final to Prevent Overriding:

While method overridingul features,is thereonewill ofbetimesJava'swhenyou most will want to prevent it from occurring. To disallow a method from being overridden, specify

final as a modifier at the start of its declaration. Methods declared as **final** cannot be overridden.

The following fragment illustrates **final**:

```
class A {  
    final void meth() {  
        System.out.println("This is a final method.");  
    }  
}
```

```
class B extends A {  
    void meth() { // ERROR! Can't  
        override. System.out.println("Illegal!");  
    }  
}
```

Using final to Prevent Inheritance:

Sometimes you will want to prevent a class from being inherited. To do this, precede the class declaration with **final**. Declaring a class as **final** implicitly declares all of its methods as **final**, too. As you might expect, it is illegal to declare a class as both **abstract** and **final** since an abstract class is incomplete by itself and relies upon its subclasses to provide complete implementations.

Here is an example of a **final** class:

```
final class A {  
    // ...  
}  
// The following class is illegal.  
class B extends A { // ERROR! Can't subclass  
    A // ...  
}
```

As the comments imply, it is illegal for **B** to inherit **A** since **A** is declared as **final**.

Polymorphism Method overriding:

- ☐ In a class hierarchy, when a method in a sub class has the same name and type signature

~~as a method in its super class, then the method in the sub class is said to be override the method in the sub class.~~

- ❑ When an overridden method is called from within a sub class, it will always refers to the version of that method defined by the sub class.
- ❑ The version of the method defined in the super class is hidden.
- ❑ In this situation, first it checks the method is existed in super class are not. If it is existed then it executes the version of sub class otherwise it gives no such method found exception.

Note: Methods with different signatures overloading but not overriding.

```
// Method
overriding. class A {
int i, j;
A(int a, int b)
{ i = a;
j = b;
}
// display i and
j void show() {
System.out.println("i and j: " + i + " " + j);
}
}
class B extends A
{ int k;
B(int a, int b, int c)
{ super(a, b);
k = c;
}
// display k –this overrides show() in A
void show() {
System.out.println("k: " + k);
}
}
class Override {
public static void main(String args[]) {
B subOb = new B(1, 2, 3);
subOb.show(); // this calls show() in B
}}
```

Output:

k: 3

Dynamic method dispatch

- o It is a mechanism by which a call to an overridden method is resolved at run time rather than compile time.
- o It is important because this is how java implements runtime polymorphism.
- o Before going to that we must know about super class reference sub class object.

```
// Dynamic Method Dispatch
class A {
void callme() {
System.out.println("Inside A's callme method");
```

```

    }
}
class B extends A {
// override callme()
void callme() {
System.out.println("Inside B's callme method");
}
}

class C extends A {
// override callme()
void callme() {
System.out.println("Inside C's callme method");
}
}

class Dispatch {
public static void main(String args[]) {
A a = new A(); // object of type A
B b = new B(); // object of type B
C c = new C(); // object of type C
A r; // obtain a reference of type A
r = a; // r refers to an A object
r.callme(); // calls A's version of callme
r = b; // r refers to a B object

r.callme(); // calls B's version of callme
r = c; // r refers to a C object r.callme();
// calls C's version of callme
}
}

```

Output:

```

Inside      A's  callme  method
Inside      B's  callme  method
Inside      C's  callme  method

```

Abstract class:

- o An abstract method is a method that is declared with only its signatures with out implementations.
- o An abstract class is class that has at least one abstract method.

The syntax is:

```

Abstract class class-name
{
Variables Abstract
methods; Concrete
methods;

```

.}

- o We can't declare any abstract constructor. o Abstract class should not include any abstract static method.
- o Abstract class can't be directly instantiated with
- o Any sub class of abstract class must be either implements all the abstract methods in the super class or declared it self as abstract.
- o Abstract modifiersubclassresponsibilitiesreferred .sebecauofasnoimplementation— of methods. Thus, a sub class must overridden them.

```
// A Simple demonstration of
abstract. abstract class A {
    abstract void callme();
// concrete methods are still allowed in abstract classes
    void callmetoo() {
        System.out.println("This is a concrete method.");
    }
}

class B extends A
{ void callme() {
    System.out.println("B's implementation of callme.");
}
}
```

```
class AbstractDemo {
    public static void main(String args[]) {
        B b = new B();
        a.
        lme();

        b.
        callme
        too();
    }
}
```

// Using abstract methods and classes.

```
abstract class Figure {
    double dim1;.

    double dim2;
    Figure(double a, double b) {
        dim1 = a;
        dim2 = b;
    }
}
```

```
// area is now an abstract method
abstract double area();
}

class Rectangle extends Figure
{ Rectangle(double a, double b)
```

```

    { super(a, b);
    }
    // override area for
    rectangle double area() {
    System.out.println("Inside Area for
    Rectangle."); return dim1 * dim2;
    }
    }
    class Triangle extends Figure {
    Triangle(double a, double b) {
    super(a, b);
    }
    // override area for right triangle
    double area() {
    System.out.println("Inside Area for
    Triangle."); return dim1 * dim2 / 2;
    }
    }

    class AbstractAreas
    {
    public static void main(String args[])
    {
    // Figure f = new Figure(10, 10); // illegal now
    Rectangle r = new Rectangle(9, 5);
    Triangle t = new Triangle(10, 8);
    Figure figref; // this is OK, no object is
    created figref = r;
    System.out.println("Area is " +
    figref.area()); figref = t;
    System.out.println("Area is " + figref.area());
    }
    }

```

Packages and Interfaces : Defining, Creating and Accessing a Package, Understanding CLASSPATH, importing packages, differences between classes and interfaces, defining an interface, implementing interface, applying interfaces, variables in interface and extending interfaces.

Defining Package:

- o Generally, any java source file contains any (or all) of the following internal parts:
 - o A single package statement (optional)
 - o
 - o Any number of import statements (optional)
 - o A single public class declaration (required)
 - o Any number of classes private to the package (optional)
- o Packages and Interfaces are two of the basic components of a java program.
- o Packages are collection of related classes.
- o Packages are containers for classes that are used to keep the class name compartmentalized.
- o Packages are stored in an hierarchical manner and are explicitly imported into new class definition.
- o Java packages are classified into two types:

~~o Java API package or pre-defined packages or built-in packages.~~

User-defined packages

o Java 2 API contains 60 java.* packages.

o For Example:

- ☐ java.lang
- ☐ Java.io
- ☐ Java.awt
- ☐ Java.util
- ☐ Java.net
- ☐ Javax.swing

- ☐ To create a package
- ☐ Just give package <<packagename>> as a first statement in java program.
- ☐ Any classes declared within that file will belong to the specified package.
- ☐ If we omit package statement, the classes are stored in the default package.
- ☐ Syntax:
- ☐ Package packagename

Syntax: Package packagename.subpackage

Access protection:

- ☐ Classes and packages both means of encapsulating and containing the name space and scope of variables and methods.
- ☐ Packages acts as a containers for classes and other sub-ordinate packages.

- ☐ Classes act as containers for data and code.
- ☐ Java address four categories of visibility for class members:
 - o Sub-classes in the same package.
 - o Non-sub class in the same package.
 - o Sub-classes in the different package.
 - o Classes that are neither in the same package nor subclasses.
- ☐ The 3 access specifiers private, public and protected provide a variety of ways to produce the many levels of access required by these categories.

Access specifier	Private	No modifier	Protected	Public
Access Location				
Same class	Yes	Yes	Yes	Yes
Same package sub class	No	Yes	Yes	Yes

Same package non-sub class	No	Yes	Yes	Yes
Different package sub class	No	No	Yes	Yes
Different package non-sub class	No	No	No	Yes

From the above table,

- o Any thing declared public can be accessed from any where
- o Any thing accessed private cannot be accessed from outside of its class
- o In the default, it is visible to sub-class as well as to other classes in the same package
- o Any thing declared as protected, this is allow an element to be seen outside your current package, but also allow to sub class in other packages access.

Importing Packages:

There are no core Java classes in the unnamed default package; all of the standard classes are stored in some named package. Since classes within packages must be fully qualified with their package name or names, it could become tedious to type in the long dot-separated package path name for every class you want to use. For this reason, Java includes the **import** statement to bring certain classes, or entire packages, into visibility. Once imported, a class can be referred to directly, using only its name. The **import** statement is a convenience to the programmer and is not technically needed to write a complete Java program. If you are going to refer to a few dozen classes in your application, however, the **import** statement will save a lot of typing.

In a Java source file, **import** statements occur immediately following the **package** statement (if it exists) and before any class definitions. This is the general form of the

import statement:

import pkg1[.pkg2].(classname)*;

Here, *pkg1* is the name of a top-level package, and *pkg2* is the name of a subordinate package inside the outer package separated by a dot (.). There is no practical limit on the depth of a package hierarchy, except that imposed by the file system. Finally,

you specify either an explicit *classname* or a star (*), which indicates that the Java compiler should import the entire package. This code fragment shows both forms in use:

```
import java.util.Date;
import java.io.*;
```

Difference between classes and interfaces:

A class is a template for an
object. (or)

A class is a way of binding variables and methods in a single unit. With the class it is possible to create object for that object. With the one class we can extend an another class.

An interface is collection of undefined method. Means all the methods are not contain any body. We have to provide the body for that interface. with the interface it is not possible to create object. For the declared interface we have to implement that interface.

Defining Interfaces:

- ☐ Interface is a collection of method declarations and constants that one or more classes of

objects will use.

- ☐ We can implement multiple inheritance using interface.
- ☐ Because interface consists only signatures followed by semi colon and parameter list they are implicitly abstract.
- ☐ Variables can be declared and initialized inside interface they are implicitly final and static.
- ☐ An interface method can't be final or stat
- ☐ An interface can be extended from another interface.
- ☐ Declaration of interface:

Access interface name

```
{
    Return type member-name1(parametelist);
    Return type member-name2(parametelist);
    .
    .
    .
    Type finalvariablename=initialization;
}
```

- o There will be no default implementation for methods specified in an interface.
- o Each class that include interface must implements all methods.
- o All the methods and variables are implicitly public if interface itself is declared as public.

Implementing Interfaces:

Once an **interface** has been defined, one or more classes can implement that interface. To implement an interface, include the **implements** clause in a class definition, and then create the methods defined by the interface. The general form of a class that includes the **implements** clause looks like this:

```
access class classname [extends superclass]
[implements interface [,interface...]] {
// class-body
}
```

Here, *access* is either **public** or not used. If a class implements more than one interface, the interfaces are separated with a comma. If a class implements two interfaces that declare the same method, then the same method will be used by clients of either interface. The methods that implement an interface must be declared **public**. Also, the type signature of the implementing method must match exactly the type signature specified in the **interface** definition.

Applying Interfaces:

To understand the power of interfaces, let chapters you developed a class called **Stack** that implemented a simple fixed-size stack.

However, there are many ways to implement a stack. For example, the stack can be of a fixed size or it can be —growable.¶ The stack can and so on. No matter how the stack is implemented, the interface to the

stack remains the same. That is, the methods **push()** and **pop()** define the interface to the stack independently of the details of the implementation. Because the interface to a stack is separate from its implementation, it is easy to define a stack interface, leaving it to each implementation to define the specific

First, here is the interface that defines an integer stack. Put this in a file called **IntStack.java**. This interface will be used by both stack implementations.

```
// Define an integer stack
interface IntStack {
    void push(int item); // store an item
    int pop(); // retrieve an item
}
```

Variables in Interfaces:

When you include that interface in a class (interface), all of those variable names will be in scope as constants. This is similar to using a header file in C/C++ to create a large number of **#defined** constants or **const**

declarations. If an interface contains no methods, then any class that includes such an interface doesn't implement anything actually. It is as if that class were importing the constant variables into the class name space as **final** variables.

```
import java.util.Random;
interface SharedConstants
{ int NO = 0;
  int YES = 1;
  int MAYBE = 2;
  int LATER = 3;
  int SOON = 4;
  int NEVER = 5;
}

class Question implements SharedConstants
{ Random rand = new Random();
  int ask() {
    int prob = (int) (100 * rand.nextDouble());
    if (prob < 30)
      return NO; // 30%
    else if (prob < 60)
      return YES; // 30%
    else if (prob < 75)
      return LATER; //
      15% else if (prob <
      98) return SOON; //
      13% else
      return NEVER; // 2%
    } }

class AskMe implements SharedConstants
{ static void answer(int result) {
  switch(result) {
    case NO:
      System.out.println("No");
      break;
    case YES:
      System.out.println("—yes!"); break;
    case MAYBE:
      System.out.println("Maybe");
```

```

break;
case LATER:
System.out.println("Later");
break;
case SOON:
System.out.println("Soon");
break;
case NEVER:
System.out.println("Never");
break;
}}
public static void main(String args[]) {
Question q = new Question();
answer(q.ask());
answer(q.ask());
answer(q.ask());
answer(q.ask());
}}

```

Interfaces Can Be Extended:

One interface can inherit another by use of the keyword **extends**. The syntax is the same as for inheriting classes. When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain. Following is an example:

```

// One interface can extend
another. interface A {
void meth1();
void meth2();
}
// B now includes meth1() and meth2() -- it adds
meth3(). interface B extends A {
void meth3();
}
// This class must implement all of A and B
class MyClass implements B {
public void meth1() {
System.out.println("Implement meth1().");
}
public void meth2() {
System.out.println("Implement meth2().");
}
public void meth3() {
System.out.println("Implement meth3().");
}
}
class IFExtend {
public static void main(String arg[]) {
MyClass ob = new MyClass();
ob.meth1();
ob.meth2();
}
}

```

```
ob.meth3();  
}  
}
```

As an experiment you might want to try removing the implementation for **meth1()** in **MyClass**. This will cause a compile-time error. As stated earlier, any class that implements an interface must implement all methods defined by that interface, including any that are inherited from other interfaces.

Unit-III

Exception Handling –Dealing with errors- benefits of exception handling- the classification of exceptions –exception hierarchy- checked exceptions and unchecked exceptions- usage of try-catch-throw-throws and finally-rethrowing exceptions- exception specification- built in exceptions- creating own exception sub classes.

Multithreading –Differences between multiple processes and multiple threads- thread states- creating threads- interrupting threads- thread priorities- synchronizing threads- inter –thread communication- producer consumer pattern

Introduction

- o An exception is an event that occurs during the execution of a program that disrupts the normal flow of instruction.

Or

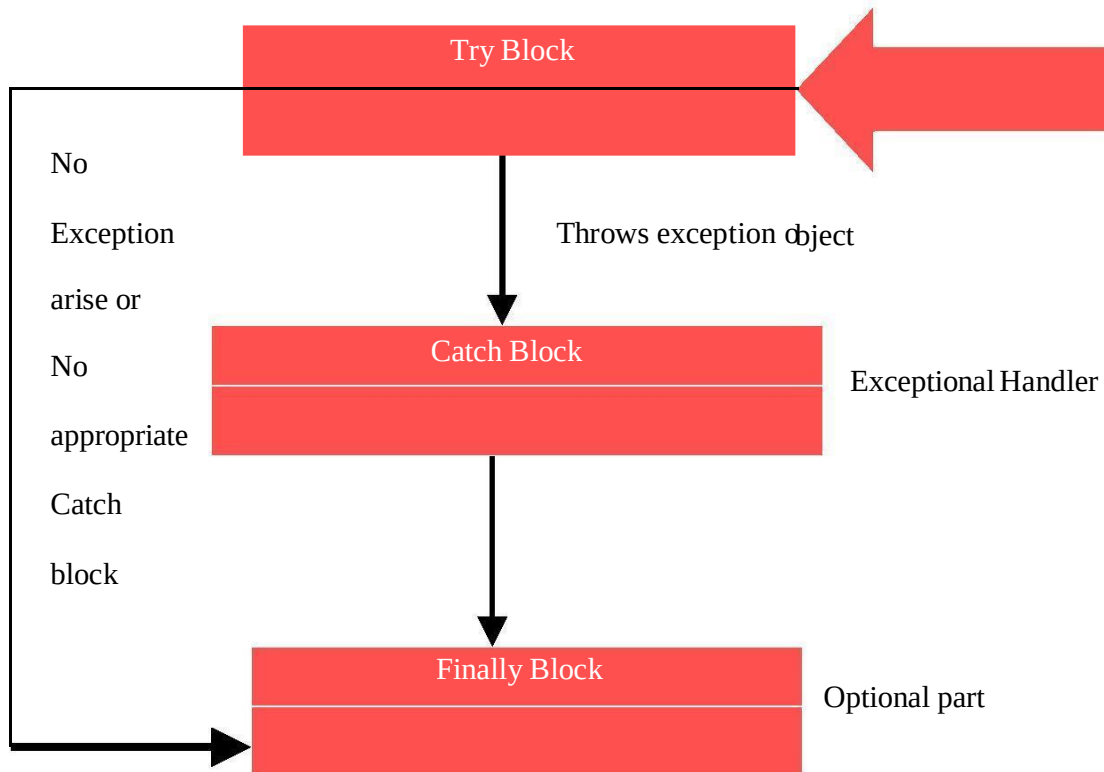
- o An abnormal condition that disrupts Normal program flow.
- o There are many cases where abnormal conditions happen during program execution, such as
 - o Trying to access an out - of –bounds array elements.
 - o The file you try to open may not exist.
 - o The file we want to load may be missing or in the wrong format.
 - o The other end of your network connection may be non –existence.
- o If these cases are not prevented or at least handled properly, either the program will be aborted abruptly, or the incorrect results or status will be produced.
- o When an error occurs with in the java method, the method creates an exception object and hands it off to the runtime system.
- o The exception object contains information about the exception including its type and the state of the program when the error occurred. The runtime system is then responsible for finding some code to handle the error.
- o In java creating an exception object and handling it to the runtime system is called throwing an exception.
- o Exception is an object that is describes an exceptional (i.e. error) condition that has occurred in a piece of code at run time.
- o When a exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error. That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is *caught* and processed.
- o Exceptions can be generated by the Java run-time system, or they can be manually generated by your code.
- o System generated exceptions are automatically thrown by the Java runtime system

General form of Exception Handling
block try {
// block of code to monitor for errors
}

```

catch (ExceptionType1 exOb) {
// exception handler for ExceptionType1
}
catch (ExceptionType2 exOb) {
// exception handler for ExceptionType2
}
// ...
finally {
// block of code to be executed before try block ends
}

```



🌟 By using exception to managing errors, Java programs have have the following advantage over traditional error management techniques:

- Separating Error handling code from regular code.
- Propagating error up the call stack.
- Grouping error types and error differentiation.

For Example:

```

class Exc0 {
public static void main(String args[]) {
int d = 0;
int a = 42 / d;
}
}

```


When the Java run-time system detects the attempt to divide by zero, it constructs a new exception object and then *throws* this exception. This causes the execution of **Exc0** to stop, because once an exception has been thrown, it must be *caught* by an exception handler and dealt with immediately. In this example, we haven't seen the exception is caught by the default handler provided by the Java run-time system. Any

exception that is not caught by your program will ultimately be processed by the default handler. The default handler displays a string describing the exception, prints a stack trace from the point at which the exception occurred, and terminates the program. Here is the output generated when this example is executed.

```
java.lang.ArithmeticException: / by
zero at Exc0.main(Exc0.java:4)
```

Notice how the class name, **Exc0**; the method name, **main**; the filename, **Exc0.java**; and the line number, **4**

Try and Catch Blocks

- ✎ If we don't want to prevent the program from the exception using the try block. So we can place the statements that may cause an exception in the try block.

```
Try
{
}
```

- ✎ If an exception occurs within the try block, the appropriate exception handler that is associated with the try block handles the exception immediately following the try block, include a catch clause specifies the exception type we wish to catch. A try block must have at least one catch block or finally that allows it immediately.

Catch block

- ✎ The catch block is used to process the exception raised. A try block can be one or more catch blocks can handle a try block.
- ✎ Catch handles are placed immediately after the try block.

```
Catch(exceptiontype e)
{
    //Error handle routine is placed here for handling exception
}
```

Program 1

```
Class trycatch
{
    Public static void main(String args[])
    {
        Int[] no={1,2,3};
        Try
        {
            System.out.println(no[3]);
        }
    }
}
```

```

}
Catch(ArrayIndexOutOfBoundsException e)
{
System.out.println(—Out of bounds!);
}
System.out.println(—Quit!);
}
}

```

Output

Out of the Range
Quit

```

Program 2 Class
ArithExce
{
Public static void main(String args[])
{
Int a=10;
Int b=0;
Try
{
a=a/b;
System.out.println(—Won't Print!);
}
Catch(ArithmeticException e)
{
System.out.println(—Division by Zero error!); System.out.println(—Change the b value!);

}
System.out.println(—Quit!);
}
}

```

Output

Division By zero error
Please change the B value
Quit

Note:

- 👉 A try and its catch statement form a unit.
- 👉 We cannot use try block alone.
- 👉 The compiler does not allow any statement between try block and its associated catch block

Displaying description of an Exception

- ✿ Throwable overrides the toString() method (defined by Object) so that it returns a string containing a description of the exception.
- ✿ We can display this description in a println statement by simply passing the exception as an argument.

```
catch (ArithmeticException e) {  
    System.out.println("Exception: " + e);  
    a = 0; // set a to zero and continue  
}
```

- ✿ When this version is substituted in the program, and the program is run, each divide-by-zero error displays the following message:
 - Exception: java.lang.ArithmeticException: / by zero

Multiple Catch Blocks

In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, you can specify two or more **catch** clauses, each catching a different type of exception. When an exception is thrown, each **catch** statement is inspected in order, and the first one whose type matches that of the exception is executed. After one **catch** statement executes, the others are bypassed, and execution continues after the **try/catch** block. The following example traps two different exception types:

```
// Demonstrate multiple catch statements.  
class MultiCatch {  
    public static void main(String args[]) {  
        try {  
            int a = args.length;  
            System.out.println("a = " + a);  
            int b = 42 / a;  
            int c[] = { 1 };  
            c[42] = 99;  
        } catch (ArithmeticException e) {  
            System.out.println("Divide by 0: " + e);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index oob: " + e);  
        }  
        System.out.println("After try/catch blocks.");  
    }  
}
```

This program will cause a division-by-zero exception if it is started with no commandline parameters, since **a** will equal zero. It will survive the division if you provide a command-line argument, setting **a** to something larger than zero. But it will cause an

ArrayIndexOutOfBoundsException, since the **int** array **c** has a length of 1, yet the program attempts to assign a value to **c[42]**.

Here is the output generated by running it both ways: C:\>java MultiCatch
a = 0

Divide by 0: java.lang.ArithmeticException: / by zero
After try/catch blocks.

C:\>java MultiCatch
TestArg a = 1

Array index oob: java.lang.ArrayIndexOutOfBoundsException
After try/catch blocks.

Throw Keyword

- ✿ So far, we have only been catching exceptions that are thrown by the Java Run –Time systems. However, it is possible for our program to throw an exception explicitly, using the throw statement.

✿ Throw ThrowableInstance

- ✿ Here, *ThrowableInstance* must be an object of type **Throwable** or a subclass of **Throwable**. Simple types, such as **int** or **char**, as well as non-**Throwable** classes, such as **String** and **Object**, cannot be used as exceptions
- ✿ There are two ways you can obtain a **Throwable** object:
 - using a parameter into a **catch** clause
 - creating one with the **new** operator.

- ✿ The flow of execution stops immediately after the **throw** statement; any subsequent statements are not executed. The nearest enclosing **try** block is inspected to see if it has a **catch** statement that matches the type of the exception. If it does find a match, control is transferred to that statement. If not, then the next enclosing **try** statement is inspected, and so on. If no matching **catch** is found, then the default exception handler halts the program and prints the stack trace

```
// Demonstrate throw.
class ThrowDemo {
static void demoproc()
{ try {
throw new NullPointerException("demo");
} catch(NullPointerException e) {
System.out.println("Caught inside
demoproc."); throw e; // rethrow the exception
}
}
public static void main(String args[]) {
try {
demoproc();
} catch(NullPointerException e) {
System.out.println("Recaught: " + e);
}
}}
```

- ✦ This program gets two chances to deal with the same error. First, **main()** sets up an exception context and then calls **demoproc()**. The **demoproc()** method then sets up another exception-handling context and immediately throws a new instance of **NullPointerException**, which is caught on the next line. The exception is then rethrown. Here is the resulting output:
- ✦ Caught inside demoproc.
- ✦ Recaught: java.lang.NullPointerException: demo
- ✦ The program also illustrates how to create one of J close attention to this line:
- ✦ `throw new NullPointerException("demo");`
- ✦ Here, **new** is used to construct an instance of **NullPointerException**. All of- Java' in run-time exceptions have at least two constructors: one with no parameter and one that takes a string parameter. When the second form is used, the argument specifies a string that describes the exception. This string is displayed when the object
- ✦ is used as an argument to **print()** or **println()**. It can also be obtained by a call to **getMessage()**, which is defined by **Throwable**.

Throws Keyword

- ✦ If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception. You do this by including a **throws** clause in the method's **throws** clause lists declare the types of exceptions that a method might throw. This is necessary for all exceptions, except those of type **Error** or **RuntimeException**, or any of their subclasses. All other exceptions that a method can throw must be declared in the **throws** clause. If they are not, a compile-time error will result. This is the general form of a method declaration that includes a **throws** clause:

```
type method-name(parameter-list) throws exception-list
{
// body of method
}
```

- ✦ Here, *exception-list* is a comma-separated list of the exceptions that a method can throw

Program

```
class ThrowsDemo {
static void throwOne() throws IllegalAccessException
{ System.out.println("Inside throwOne.");
throw new IllegalAccessException("demo");
}
public static void main(String args[]) {
try {
throwOne();
} catch (IllegalAccessException e)
{ System.out.println("Caught " + e);
}
}
}
```

👉 Here is the output generated by running this example program:

👉 inside throwOne

👉 caught java.lang.IllegalAccessException

S.No. throw

throws

- | | | |
|----|--|---|
| 1) | Java throw keyword is used to explicitly throw an exception. | Java throws keyword is used to declare an exception. |
| 2) | Checked exception cannot be propagated throws. | Checked exception can be propagated with using throw only. |
| 3) | Throw is followed by an instance. | Throws is followed by class. |
| 4) | Throw is used within the method. | Throws is used with the method signature. |
| 5) | You cannot throw multiple exceptions. | You can declare multiple exceptions e.g.
public void method()throws
IOException,SQLException. |

Finally block

- 👉 When exceptions are thrown, execution in a method takes a rather abrupt, nonlinear path that alters the normal flow through the method. Depending upon how the method is coded, it is even possible for an exception to cause the method to return prematurely. This could be a problem in some methods. For example, if a method
- 👉 opens a file upon entry and closes it upon exit, then you will not want the code that closes the file to be bypassed by the exception-handling mechanism. The **finally** keyword is designed to address this contingency.
- 👉 **finally** creates a block of code that will be executed after a **try/catch** block has completed and before the code following the **try/catch** block. The **finally** block will execute whether or not an exception is thrown. If an exception is thrown, the **finally** block will execute even if no **catch** statement matches the exception. Any time a method is about to return to the caller from inside a **try/catch** block, via an uncaught exception or an explicit return statement, the **finally** clause is also executed just before the method returns. This can be useful for closing file handles and freeing up any other resources that

might have been allocated at the beginning of a method with the intent of disposing of them before returning. The **finally** clause is optional. However, each **try** statement requires at least one **catch** or a **finally** clause.

```
// Demonstrate finally.
class FinallyDemo {
// Through an exception out of the method.
static void procA() {
try {
System.out.println("inside procA");
throw new RuntimeException("demo");
} finally {
System.out.println("procA's finally");
}
}

// Return from within a try
block. static void procB() {
try {
System.out.println("inside procB");
return;
} finally { System.out.println("procB's
finally");
}
}

// Execute a try block normally.
static void procC() {
try {
System.out.println("inside procC"); }
finally { System.out.println("procC's
finally");
}
}

public static void main(String args[]) {
try {
procA();
} catch (Exception e) {
System.out.println("Exception caught");
}
procB();
procC();
}
}
```

🍌 In this example, **procA()** prematurely breaks out of the **try** by throwing an exception. The **finally** clause is executed on the way out. **procB()**'s **try** statement is exited via a **return** statement. The **finally** clause is executed before **procB()** returns. In **procC()**, the

try statement executes normally, without error. However, the **finally** block is still executed. *If a **finally** block is associated with a **try**, the **finally** block will be executed upon conclusion of the **try**.*

👉 Here is the output generated by the preceding

```
program: inside procA
procA's      finally
Exception caught
inside procB
procB's finally inside procC

procC'sly final
```

Difference between final, finally and finalize

There are many differences between final, finally and finalize. A list of differences between final, finally and finalize are given below:

No.	final	finally	finalize
	Final is used to apply restrictions on class, method and variable. Final class important code, it will be perform	Finally is used to place clean up	Finalize is used to
1)	can't be inherited, final method can't be executed overridden and final variable value can't be changed.	whether processing just before exception is handled or object is garbage collected.	
2)	Final is a keyword.	Finally is a block.	Finalize is a method.

Java final example

```
1. class FinalExample{
2.   public static void main(String[] args){
3.     final int x=100;
4.     x=200;//Compile Time Error
5. }}
```

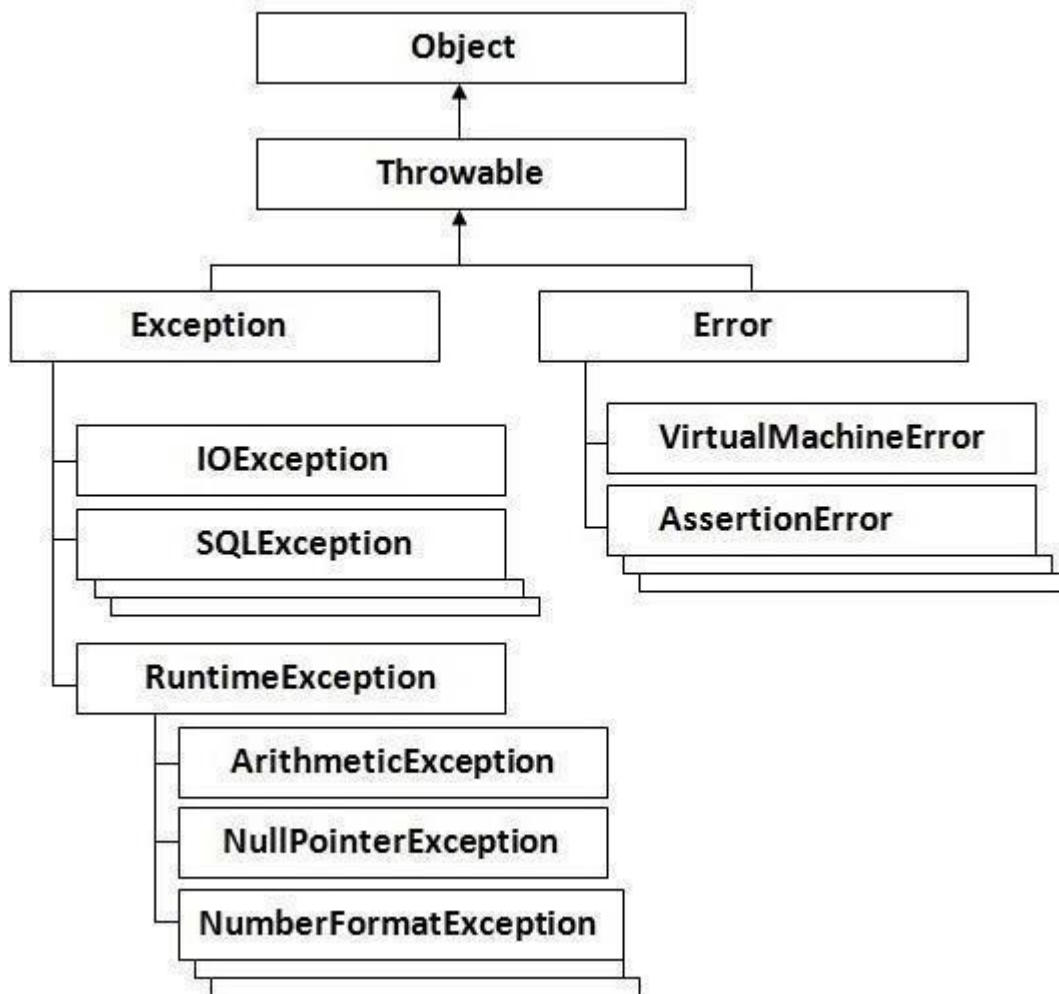
Java finally example

```
1. class FinallyExample{
2.   public static void main(String[] args){
3.     try{
4.       int x=300;
5.     }catch(Exception e){System.out.println(e);}
6.     finally{System.out.println("finally block is executed");}
7. }}
```


Java finalize example

```
1. class FinalizeExample{
2.     public void    finalize(){System.out.println("finalize
   called");}
3.     public static void main(String[] args){
4.         FinalizeExample f1=new FinalizeExample();
5.         FinalizeExample f2=new FinalizeExample();
6.         f1=null;
7.         f2=null;
8.         System.gc();
9.     }}
```

Hierarchy of Java Exception classes



Java Built-In Exceptions

Inside the standard package **java.lang**, Java defines several exception classes. A few have been used by the preceding examples. The most general of these exceptions are subclasses of the standard type **RuntimeException**. Since **java.lang** is implicitly imported into all Java programs, most exceptions derived from **RuntimeException** are automatically available. Furthermore, they need not be included in the **throws** list. In the language any of a method's Java, these are called *unchecked exceptions* because the compiler does not check to see if a method handles or throws these exceptions. The unchecked exceptions defined in **java.lang** are listed in Table 10-1. Table

lists those exceptions defined by **java.lang** that must be included in the **throws** list in if that method can generate one of these exceptions and does not handle it itself. These are called *checked exceptions*. Java defines several other types of exceptions that relate to its various class libraries

List of Unchecked exceptions

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
Exception	Meaning
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
UnsupportedOperationException	An unsupported operation was encountered.

List of Checked exceptions

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

User defined exceptions

- ✦ We can create our own exception by extending exception class.
- ✦ The throw and throws keywords are used while implementing user defined exceptions

Class ownException extends Exception

```
{  
ownException(String msg)  
{  
Super(msg);  
}  
}
```

Class test

```
{  
Public static void main(String args[])  
Int mark=101;  
Try  
{  
if(mark>100)  
{  
Throw new ownException("Marks>100");  
}  
}  
Catch(ownException e)  
{  
System.out.println("Exception caught"); System.out.println(e.getMessage());  
}  
Finally  
{  
System.out.println("End of prg");  
}  
}
```

Output:

Exception caught

Marks is > 100

End of program

Multi Threaded Programming

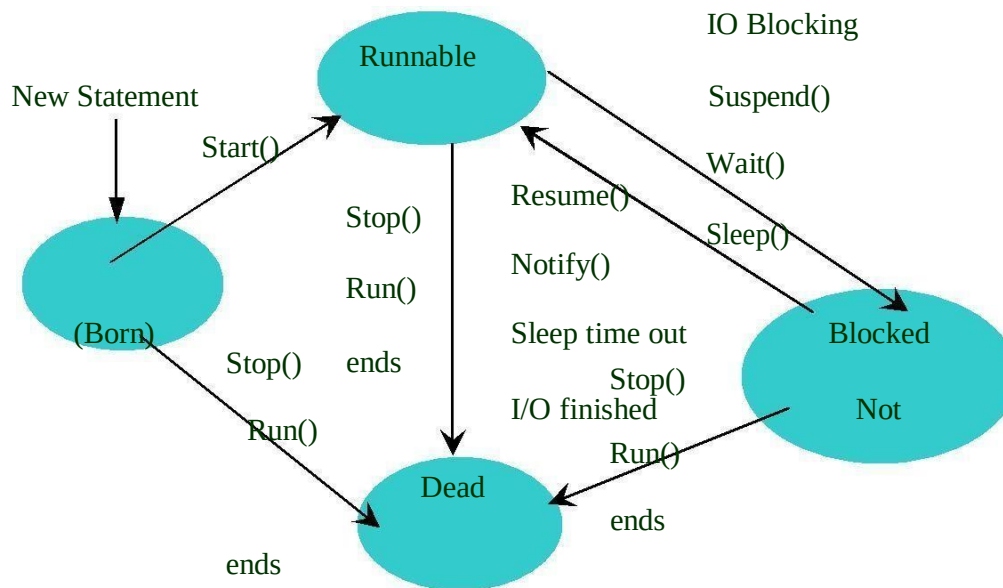
Introduction:

- o Java provides a built –in support for multithreaded programming.
- o A multithreaded program contains two or more parts that can run concurrently.
- o Each part of such a program called thread.
- o Each thread defines a separate path of execution.
- o Thus multi thread is a specialized form of multi tasking.
- o Multi tasking is supported by OS
- o There are two distinct types of multi tasking
- o Process based multi tasking
- o Process is a program that is executing.
- o In process based multi tasking, a program is the smallest unit of code that can be dispatched by the scheduler
- o Process based multi tasking is a feature that allows computer to run two or more programs concurrently
- o For example :
- o This tasking enables us to run the Java compiler and text editor at the same time
- o Thread based multi tasking
- o Thread is a smallest unit of dispatchable code
- o The single program can perform two or more tasks simultaneously.
- o For example:
- o A text editor can format text at the same time that is printing as long as these two actions are performed by two separate threads.
- o Multitasking threads require less overhead than multitasking processes.

Thread Model

- o One thread can pause without stopping other parts of your program.
 - o For example, the idle time created when a thread reads data from a network or waits for user input can be utilized elsewhere.
 - o Multithreading allows animation loops to sleep for a second between each frame without causing the whole system to pause.
- o When a thread blocks in a Java program, only the single thread that is blocked pauses. All other threads continue to run.
- o Thread States
 - o Threads exist in several states.
 - ☐ A thread can be *running*. It can be *ready to run* as soon as it gets CPU time.
 - ☐ A running thread can be *suspended*, which temporarily suspends its activity.
 - ☐ A suspended thread can then be *resumed*, allowing it to pick up where it left off.
 - ☐ A thread can be *blocked* when waiting for a resource.

- ☐ At any time, a thread can be terminated, which halts its execution immediately.
- ☐ Once terminated, a thread cannot be resumed
- o Every thread after creation and before destruction can have any one of four states:
 - Newly created
 - o Runnable state
 - o Blocked
 - o Dead
 - o



THREAD LIFE CYCLE

New State

- o A thread enters the newly created by using a new operator.
- o It is new state or born state immediately after creation. i.e. when a constructor is called the Thread is created but is not yet to run() method will not begin until it start() method is called.
- o After the start() method is called, the thread will go to the next state, Runnable state.
- o Note : in the above cycle stop(), resume() and suspend are deprecated methods. Java 2 strongly discourage their usage in the program
- o **Runnable state**
 - ☐ Once we invoke the start() method, the thread is runnable.
 - ☐ It is divided into two states:
 - ☐ The running state
 - ☐ When the thread is running state, it assigned by CPU cycles and is actually running.
 - ☐ The Queued state.
 - ☐ When the thread is in Queued state, it is waiting in the Queue and competing for its turn to spend CPU cycles
 - ☐ It is controlled by Virtual Machine Controller.
 - ☐ When we use yield() method it makes sure other threads of the same priority have chance to run.
 - ☐ This method cause voluntary move itself to the queued state from the running state.

Blocked state

- The blocked state is entered when one of the following events occurs:
 - The thread itself or another thread calls the `suspend()` method (it is deprecated)
 - The thread calls an object's `wait()` method
 - The thread itself calls the `sleep()` method.
 - The thread is waiting for some I/O operations to complete.
 - The thread will `join()` another thread.

Dead state

- A thread is dead for any one of the following reasons:
 - It dies a natural death because the `run` method exists normally.
 - It dies abruptly because an uncaught exception terminates the `run` method.
 - In particular `stop()` is used to kill the thread. This is deprecated.
 - To find whether thread is alive i.e. currently running or blocked
 - Use `isAlive()` method
 - If it returns `true` the thread is alive

Thread priorities

- Java assigns to each thread a priority that determines how that thread should be treated with respect to the others. Thread priorities are integers that specify the relative priority of one thread to another. As an absolute value, a priority is meaningless; a higher-priority thread doesn't run-priorityanythread fasterifitistheonly threadthanrunninga. lower Instead, a thread's priority is used to determine the next. This is called a *context switch*. The rules that determine when a context switch takes place are simple:
 - *A thread can voluntarily relinquish control.* This is done by explicitly yielding, sleeping, or blocking on pending I/O. In this scenario, all other threads are examined, and the highest-priority thread that is ready to run is given the CPU.
 - *A thread can be preempted by a higher-priority thread.* In this case, a lower-priority thread that does not yield the processor is simply preempted—no matter what it is doing—by a higher-priority thread. Basically, as soon as a higher-priority thread wants to run, it does. This is called *preemptive multitasking*.
- In cases where two threads with the same priority are competing for CPU cycles, the situation is a bit complicated. For operating systems such as Windows 98, threads of equal priority are time-sliced automatically in round-robin fashion. For other types of operating systems, threads of equal priority must voluntarily yield control to their peers. If they don't, the other threads will not

Synchronization

- Because multithreading introduces an asynchronous behavior to your programs, there must be a way for you to enforce synchronicity when you need it. For example, if you want two threads to communicate and share a complicated data structure, such as a linked list, you need some way to ensure that

is, you must prevent one thread from writing data while another thread is in the middle of reading it. For this purpose, Java implements an elegant twist on an age-old model of interprocess synchronization: the *monitor*. The monitor is a control mechanism first defined by C.A.R. Hoare. You can think of a monitor as a very small box that can hold only one thread. Once a thread enters a monitor, all other threads must wait until that thread exits the monitor. In this way, a monitor can be used to protect a shared asset from being manipulated by more than one thread at a time.

Messaging



After you divide your program into separate threads, you need to define how they will communicate with each other. When programming with most other languages, you must depend on the operating system to establish communication between threads. This, of course, adds overhead. By contrast, Java provides a clean, low-cost way for two or more threads to talk to each other, via calls to pre messaging system allows a thread to enter a synchronized method on an object, and then wait there until some other thread explicitly notifies it to come out.

Thread class and Runnable interface

The Thread Class and the Runnable Interface

Thread class, its methods, and its companion interface, **Runnable**. **Thread** encapsulates a thread of execution. Since the thread is a running thread, directly you will deal with it through its proxy, the **Thread** instance that spawned it. To create a new thread, your program will either extend **Thread** or implement the **Runnable** interface.



The **Thread** class defines several methods that help manage threads.

Method	Meaning
<code>getName</code>	Obtain a thread's name.
<code>getPriority</code>	Obtain a thread's priority.
<code>isAlive</code>	Determine if a thread is still running.
<code>join</code>	Wait for a thread to terminate.
<code>run</code>	Entry point for the thread.
<code>sleep</code>	Suspend a thread for a period of time.
<code>start</code>	Start a thread by calling its run method.

Main method

- ☐ When a Java program starts up, one thread begins running immediately. This is usually called the *main thread* of your program, because it is the one that is executed when your program begins. The main thread is important for two reasons:
 - o It is the thread from which other threads are spawned.
 - o Often it must be the last thread to finish execution because it performs various shutdown actions.
- ☐ Although the main thread is created automatically when your program is started, it can be controlled through a **Thread** object. To do so, you must obtain a reference to it by calling the method **currentThread()**, which is a **public static** member of **Thread**. Its general form is
 - o `static Thread currentThread()`
- ☐ This method returns a reference to the thread in which it is called. Once you have a reference to the main thread, you can control it just like any other thread.

// Controlling the main Thread.

```
class CurrentThreadDemo {
public static void main(String args[]) {
Thread t = Thread.currentThread();
System.out.println("Current thread: " + t);
// change the name of the thread
t.setName("My Thread");
System.out.println("After name change: " +
t); try {
for(int n = 5; n > 0; n--) {
System.out.println(n);
Thread.sleep(1000);
}
} catch (InterruptedException e) {
System.out.println("Main thread interrupted");
}
}}
Current thread: Thread[main,5,main]
After name change: Thread[My
Thread,5,main] 5 4 3 2 1
```

How to create a thread

- ☐ In the most general sense, you create a thread by instantiating an object of type **Thread**.
- ☐ Java defines two ways in which this can be accomplished:
 - ☐ You can implement the **Runnable** interface.
 - ☐ You can extend the **Thread** class, itself.

Implementing thread class

- ☐ The easiest way to create a thread is to create a class that implements the **Runnable** interface. **Runnable** abstracts a unit of executable code. You can construct a thread on any object that implements **Runnable**. To implement **Runnable**, a class need only implement a single method called **run()**, which is declared like this:
 - o `public void run()`
- ☐ Inside **run()**, you will define the code that constitutes the new thread. It is important to understand that **run()** can call other methods, use other classes, and declare variables, just like the main thread can. The only difference is that **run()** establishes the entry point for another, concurrent thread of execution within your program. This thread will end when **run()** returns.
- ☐ After you create a class that implements **Runnable**, you will instantiate an object of type **Thread** from within that class. **Thread** defines several constructors. The one that we will use is shown here:
 - o `Thread(Runnable threadOb, String threadName)`
- ☐ In this constructor, *threadOb* is an instance of a class that implements the **Runnable** interface. This defines where execution of the thread will begin. The name of the new thread is specified by *threadName*.
- ☐ After the new thread is created, it will not start running until you call its **start()**
- ☐ method, which is declared within **Thread**. In essence, **start()** executes a call to **run()**.
- ☐ The **start()** method is shown here:
 - o `void start()`

Extending thread class

The second way to create a thread is to create a new class that extends **Thread**, and then to create an instance of that class. The extending class must override the **run()** method, which is the entry point for the new thread. It must also call **start()** to begin execution of the new thread. Here is the preceding program rewritten to extend **Thread**

```
// Create a second thread by extending Thread
class NewThread extends Thread {
    NewThread() {
        // Create a new, second thread
        super("Demo Thread");
        System.out.println("Child thread: " + this);
        start(); // Start the thread
    }
    // This is the entry point for the second thread.
    public void run() {
        try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Child Thread: " + i);
                Thread.sleep(500);
            }
        } catch (InterruptedException e) {
            System.out.println("Child interrupted.");
        }
        System.out.println("Exiting child thread.");
    }
}

class ExtendThread {
    public static void main(String args[]) {
        new NewThread(); // create a new
        thread try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Main Thread: " +
                i); Thread.sleep(1000);
            }
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }
        System.out.println("Main thread exiting.");
    }
}
```

Synchronization

When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time. The process by which this is achieved is called *synchronization*. As you will see, Java provides unique, language-level support for it. Key to synchronization is the concept of the monitor (also called a *semaphore*).

A *monitor* is an object that is used as a mutually exclusive lock, or *mutex*. Only one thread can own a monitor at a given time. When a thread acquires a lock, it is said to have *entered* the monitor. All other threads attempting to enter the locked monitor will be suspended until the first thread *exits* the monitor. These other threads are said to be *waiting* for the monitor. A thread that owns a monitor can reenter the same monitor if it so desires.

You can synchronize your code in either of two ways. Both involve the use of the **synchronized** keyword, and both are examined here.

Using Synchronized Methods

Synchronization is easy in Java, because all objects have their own implicit monitor associated with them. To enter an object's monitor with the **synchronized** keyword. While a thread is inside a synchronized method, all other

threads that try to call it (or any other synchronized method) on the same instance have to wait. To exit the monitor and relinquish control of the object to the next waiting thread, the owner of the monitor simply returns from the synchronized method.

While creating **synchronized** methods within classes that you create is an easy and effective means of achieving synchronization, it will not work in all cases. To understand why, consider the following. Imagine that you want to synchronize access to objects of a class that was not designed for multithreaded access. That is, the class does not use **synchronized** methods. Further, this class was not created by you, but by a third party, and you do not have access to the source code. Thus, you can't **synchronized** add to the appropriate methods within the class. How can access to an object of this class be synchronized? Fortunately, the solution to this problem is quite easy: You simply put calls to the methods defined by this class inside a **synchronized** block.

This is the general form of the **synchronized** statement:

```
synchronized(object) {  
    // statements to be synchronized  
}
```

Here, *object* is a reference to the object being synchronized. A synchronized block ensures that a call to a method that is a member of *object* occurs only after the current thread has successfully entered *object*'s monitor.

Here is an alternative version of the preceding example, using a synchronized block within the **run()** method:

// This program uses a synchronized block.

```
class Callme {  
    void call(String msg) {  
        System.out.print("[ " +  
            msg); try {  
            Thread.sleep(1000);  
        } catch (InterruptedException e) {  
            System.out.println("Interrupted");  
        }  
        System.out.println("]");  
    }  
}  
  
class Caller implements Runnable  
{ String msg;  
  Callme target;  
  Thread t;  
  public Caller(Callme targ, String s)  
  { target = targ;  
    msg = s;  
    t = new Thread(this);  
    t.start();  
  }  
}
```

```

// synchronize calls to
call() public void run() {
synchronized(target) { // synchronized
block target.call(msg);
}
}
}

class Synch1 {
public static void main(String args[]) {
Callme target = new Callme();
Caller ob1 = new Caller(target, "Hello");
Caller ob2 = new Caller(target, "Synchronized");
Caller ob3 = new Caller(target, "World");

// wait for threads to
end try {
ob1.t.join();
ob2.t.join();
ob3.t.join();
} catch (InterruptedException e) {
System.out.println("Interrupted");
}
}
}

```

Here, the **call()** method is not modified by **synchronized**. Instead, the **synchronized** statement is used inside **Caller's run()** method. This causes the same correct output as the preceding example, because each thread waits for the prior one to finish before proceeding.

Daemon Threads

A —daemon thread is one that is supposed to as the program is running, but is not part of the essence of the program. Thus when all of the non-daemon threads complete, the program is terminated. you can find out if a thread is a daemon

by calling **isDaemon()**, and you can turn the **setDaemon()**.if a thread is a daemon, then any threads it creates will automatically be daemons.

INTER-THREAD COMMUNICATION IN JAVA

Inter-thread communication or **Co-operation** is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of **Object class**:

- ☐ **wait()**
- ☐ **notify()**
- ☐ **notifyAll()**

1) wait() method

Causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

Method

public final void wait()throws InterruptedException

public final void wait(long timeout)throws InterruptedException

Description

waits until object is notified.

waits for the specified amount of time.

2) notify() method

Wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation. Syntax:

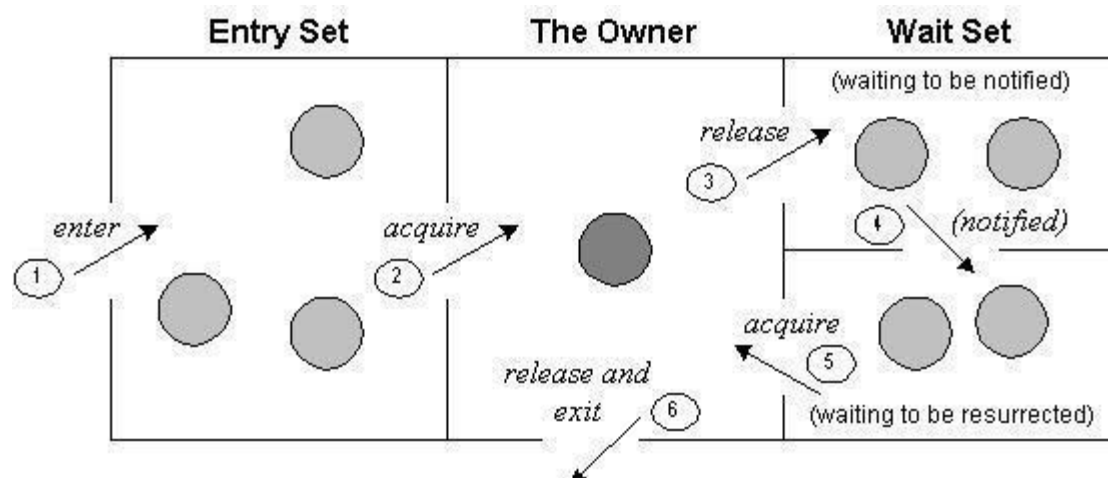
public final void notify()

3) notifyAll() method

Wakes up all threads that are waiting on this object's monitor. Syntax:

public final void notifyAll()

Understanding the process of inter-thread communication



The point to point explanation of the above diagram is as follows:

1. Threads enter to acquire lock.
2. Lock is acquired by one thread.
3. Now thread goes to waiting state if you call wait() method on the object. Otherwise it releases the lock and exits.

4. If you call notify() or notifyAll() method, thread moves to the notified state (runnable state).
5. Now thread is available to acquire lock.
6. After completion of the task, thread releases the lock and exits the monitor state of the object.

Why wait(), notify() and notifyAll() methods are defined in Object class not Thread class?

It is because they are related to lock and object has a lock.

Difference between wait and sleep?

Let's see the important differences between wait and sleep methods.

wait()

wait() method releases the lock
is the method of Object class
is the non-static method
is the non-static method
should be notified by notify() or notifyAll() after the specified amount of time, sleep is methods

sleep()

sleep() method doesn't release the lock.
is the method of Thread class
is the static method
is the static method
completed.

Example of inter thread communication in java

Let's see the simple example of inter thread communication.

1. class Customer{
2. int amount=10000;
- 3.
4. synchronized void withdraw(int amount){
5. System.out.println("going to withdraw...");
- 6.
7. if(this.amount<amount){
8. System.out.println("Less balance; waiting for deposit...");
9. try{wait();}catch(Exception e){}
10. }
11. this.amount-=amount;
12. System.out.println("withdraw completed...");
13. }
- 14.
15. synchronized void deposit(int amount){
16. System.out.println("going to deposit...");
17. this.amount+=amount;
18. System.out.println("deposit completed... ");
19. notify();
20. }
21. }

```

22.
23. class Test{
24. public static void main(String args[]){
25. final Customer c=new Customer();
26. new Thread(){
27. public void run(){c.withdraw(15000);}
28. }.start();
29. new Thread(){
30. public void run(){c.deposit(10000);}
31. }.start();
32.
33. }}

```

Output: going to withdraw...
 Less balance; waiting for deposit...
 going to deposit...
 deposit completed...
 withdraw completed

INTERRUPTING A THREAD:

If any thread is in sleeping or waiting state (i.e. `sleep()` or `wait()` is invoked), calling the `interrupt()` method on the thread, breaks out the sleeping or waiting state throwing `InterruptedException`. If the thread is not in the sleeping or waiting state, calling the `interrupt()` method performs normal behaviour and doesn't interrupt the thread but sets the interrupt flag to true. Let's first see the methods provided by the `Thread` class for thread interruption.

The 3 Methods Provided By The Thread Class For Interrupting A Thread

```

public void interrupt()

public static boolean interrupted()

public boolean isInterrupted()

```

Example of interrupting a thread that stops working

In this example, after interrupting the thread, we are propagating it, so it will stop working. If we don't want to stop the thread, we can handle it where `sleep()` or `wait()` method is invoked. Let's first see the example where we are propagating the exception.

```

1. class TestInterruptingThread1 extends Thread{
2. public void run(){
3. try{
4. Thread.sleep(1000);
5. System.out.println("task");
6. }catch(InterruptedException e){
7. throw new RuntimeException("Thread
interrupted..." + e); 8. }
9.
10. }

```